

SQL For SDET

Part 1: Core Fundamentals

★ Topics 1 to 18 — Foundation to DML ★



Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community ■

Bengaluru, Karnataka, India

[linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)

JSPM University Pune

SQL Basics	SQL Data Types	SELECT Statement	WHERE Clause
Operators in SQL	ORDER BY Clause	DISTINCT Keyword	LIMIT Clause
Aliases in SQL	INSERT Statement	UPDATE Statement	DELETE Statement
TRUNCATE	DROP Statement	ALTER TABLE	NULL Handling
SQL Constraints	Primary Key vs FK		

Introduction

This is **Part 1** of the SQL For SDET Complete Guide covering **Topics 1–18**. These are the core foundation topics asked in every SDET interview — from SQL basics to DML operations, NULL handling, and schema design.

Complete Part 1 before moving to Part 2 (Joins, Advanced Queries, Indexes, Stored Procedures, and Transactions).

Topic Group	Coverage
SQL Basics & Data Types	Foundation of all SQL work
SELECT, WHERE, Operators	Data retrieval & filtering
ORDER BY, DISTINCT, LIMIT, Aliases	Query enhancement
INSERT, UPDATE, DELETE, TRUNCATE, DROP, ALTER	DML & DDL
NULL Handling, Constraints, Keys	Data integrity

Table of Contents

PART 1 — SQL CORE FUNDAMENTALS (Topics 1–18)

1. SQL Basics ★
2. SQL Data Types ★
3. SELECT Statement ★
4. WHERE Clause ★
5. Operators in SQL ★
6. ORDER BY Clause ★
7. DISTINCT Keyword ★
8. LIMIT Clause ★
9. Aliases in SQL ★
10. INSERT Statement ★
11. UPDATE Statement ★
12. DELETE Statement ★
13. TRUNCATE Statement ★
14. DROP Statement ★
15. ALTER TABLE ★
16. NULL Handling ★
17. SQL Constraints ★
18. Primary Key vs Foreign Key ★

SQL BASICS

1. SQL Basics ★

Overview

SQL is the FOUNDATION of database testing and backend validation. Asked in every SDET, product company, API testing, and backend validation interview.

Topics Covered

- What is SQL
- Why SQL is Important ★
- Types of SQL Commands ★
- SELECT Statement ★
- WHERE Clause ★
- ORDER BY ★
- DISTINCT ★
- LIMIT ★
- Aliases ★
- Real-Time Testing Scenarios ★

► What is SQL

What is it: SQL (Structured Query Language) is used to interact with databases.

Key Components:

- Querying data
- Updating records
- Managing tables
- Database validation

How YOU should answer: SQL is a database language used to store, retrieve, update, and manage relational database data.

```
SQL
```

```
SELECT * FROM users;
```

► Why SQL is Important for SDET

What is it: SQL helps validate backend data and API responses.

Key Components:

- Database verification
- API response validation
- Data integrity testing

How YOU should answer: SQL is important for validating backend systems and ensuring data consistency during testing.

```
SQL
-- Validate API response against DB
SELECT * FROM orders WHERE order_id = 101;
```

► Types of SQL Commands

What is it: Different SQL command categories.

Key Components:

- DDL — CREATE, ALTER, DROP
- DML — INSERT, UPDATE, DELETE
- DQL — SELECT
- TCL — COMMIT, ROLLBACK
- DCL — GRANT, REVOKE

How YOU should answer: SQL commands are categorized as DDL for schema, DML for data, DQL for querying, TCL for transactions, and DCL for access.

Common Mistakes

- ✗ Confusing SQL with database itself
- ✗ Mixing command categories
- ✗ Ignoring backend validation

Expected Interview Questions

- What is SQL?
- Types of SQL commands?
- Difference between DDL and DML?
- Why SQL important for SDET?

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP	Schema management
DML	INSERT, UPDATE, DELETE	Data manipulation
DQL	SELECT	Data querying
TCL	COMMIT, ROLLBACK	Transaction control
DCL	GRANT, REVOKE	Access control

SQL DATA TYPES

2. SQL Data Types ★

Overview

Understanding SQL Data Types is VERY important for database validation, backend testing, API response verification, and production database understanding.

Topics Covered

- What are SQL Data Types
- Numeric Data Types ★
- Character/String Data Types ★
- Date & Time Data Types ★
- Boolean Data Types ★
- NULL Values ★
- Choosing Correct Data Type ★

► Numeric Data Types

What is it: Integer and decimal number storage types.

Key Components:

- INT — whole numbers
- BIGINT — large numbers
- DECIMAL — precise decimals
- FLOAT — approximate decimals

How YOU should answer: Numeric data types store integer and decimal numeric values.

```
SQL
age INT,
salary DECIMAL(10,2),
score FLOAT
```

► Character / String Data Types

What is it: Text and string storage types.

Key Components:

- CHAR — fixed-length strings
- VARCHAR — variable-length strings
- TEXT — large text data

How YOU should answer: Character data types store textual information efficiently.

```
SQL
name VARCHAR(100),
code CHAR(10),
description TEXT
```

► Date & Time Data Types

What is it: Temporal data storage types.

Key Components:

- DATE — date only
- TIME — time only
- DATETIME — date and time
- TIMESTAMP — auto time tracking

How YOU should answer: Date and time data types manage temporal information for transactions and records.

SQL

```
created_at DATETIME,  
updated_at TIMESTAMP
```

► **NULL Values**

What is it: NULL represents missing or unknown data.

Key Components:

- Unknown value
- Empty state
- Requires IS NULL to check

How YOU should answer: NULL represents missing or unavailable data. Never use = NULL; always use IS NULL.

SQL

```
SELECT * FROM users WHERE phone IS NULL;
```

Common Mistakes

- ✗ Wrong datatype selection
- ✗ Confusing NULL with empty string
- ✗ Using = NULL instead of IS NULL

Expected Interview Questions

- What are SQL Data Types?
- CHAR vs VARCHAR?
- FLOAT vs DECIMAL?
- What is NULL?

Data Type	Category	Usage
INT	Numeric	Whole numbers
DECIMAL	Numeric	Precise decimals
VARCHAR	String	Variable text
CHAR	String	Fixed text

DATE	Date/Time	Date only
BOOLEAN	Boolean	True/False

SELECT STATEMENT

3. SELECT Statement ★

Overview

SELECT is one of the MOST IMPORTANT SQL concepts. Fetching database records, API response validation, backend testing, and data verification.

Topics Covered

- What is SELECT Statement
- Selecting All Columns ★
- Selecting Specific Columns ★
- Using WHERE with SELECT ★
- DISTINCT with SELECT ★
- ORDER BY with SELECT ★
- LIMIT with SELECT ★
- Real-Time Testing Scenarios ★

► What is SELECT

What is it: SELECT statement is used to retrieve data from database tables.

Key Components:

- Data retrieval
- Column selection
- Filtering support

How YOU should answer: SELECT statement retrieves records from database tables for querying and validation.

SQL

```
SELECT * FROM employees;  
SELECT name, salary FROM employees;
```

► Selecting Specific Columns

What is it: Retrieve only required columns.

Key Components:

- Optimized queries
- Better readability

- Performance improvement

How YOU should answer: Selecting specific columns improves query performance and readability.

```
SQL
SELECT name, salary, department FROM employees;
```

► Real-Time Testing Scenarios

What is it: SQL SELECT for validation.

Key Components:

- API Validation
- Banking App
- E-Commerce
- Reporting

How YOU should answer: SELECT is used to validate API responses against DB records and verify transactions.

```
SQL
-- Validate API response
SELECT * FROM orders WHERE order_id = 101 AND status = 'ACTIVE';
```

Common Mistakes

- ✗ Using SELECT * unnecessarily
- ✗ Missing WHERE conditions
- ✗ Not validating against DB

Expected Interview Questions

- What is SELECT statement?
- Why avoid SELECT *?
- Real-time validation queries?

WHERE CLAUSE

4. WHERE Clause ★

Overview

WHERE clause is one of the MOST IMPORTANT SQL concepts for data filtering, backend validation, and API response verification.

Topics Covered

- What is WHERE Clause
- Comparison Operators ★

- Logical Operators ★
- IN Operator ★
- BETWEEN Operator ★
- LIKE Operator ★
- IS NULL Operator ★

► What is WHERE Clause

What is it: WHERE clause filters records based on specified conditions.

Key Components:

- Conditional filtering
- Data validation
- Operator support

How YOU should answer: WHERE clause filters records in SELECT, UPDATE, and DELETE queries based on given conditions.

SQL

```
SELECT * FROM employees WHERE salary > 50000;
```

► Comparison Operators

What is it: Operators used for comparing values.

Key Components:

- = Equal
- != Not Equal
- > Greater than
- < Less than
- >= Greater or equal
- <= Less or equal

How YOU should answer: Comparison operators filter rows by comparing column values with specified conditions.

SQL

```
SELECT * FROM users WHERE age >= 18;  
SELECT * FROM orders WHERE status != 'CANCELLED';
```

► Logical Operators

What is it: Combine multiple conditions.

Key Components:

- AND — both conditions true
- OR — either condition true
- NOT — negate condition

How YOU should answer: Logical operators combine multiple filter conditions for complex data retrieval.

SQL

```
SELECT * FROM employees WHERE department = 'IT' AND salary > 60000;
```

► IN Operator

What is it: Check if value exists in a list.

Key Components:

- Multiple value check
- Cleaner than multiple OR

How YOU should answer: IN operator checks if a value matches any value in a specified list.

SQL

```
SELECT * FROM employees WHERE department IN ('IT','HR','Finance');
```

► BETWEEN Operator

What is it: Filter within a range.

Key Components:

- Inclusive range
- Works with numbers and dates

How YOU should answer: BETWEEN filters records within a specified range, inclusive of both boundaries.

SQL

```
SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

► LIKE Operator

What is it: Pattern matching for string data.

Key Components:

- % — any sequence of characters
- _ — single character

How YOU should answer: LIKE operator is used for pattern matching in string columns.

SQL

```
SELECT * FROM users WHERE name LIKE 'K%';  
SELECT * FROM users WHERE email LIKE '%@gmail.com';
```

Common Mistakes

- ✗ Missing conditions in UPDATE/DELETE
- ✗ Using = NULL instead of IS NULL
- ✗ Wrong LIKE pattern

Expected Interview Questions

- What is WHERE clause?

- IN vs BETWEEN?
- LIKE operator usage?
- AND vs OR?

OPERATORS IN SQL

5. Operators in SQL ★

Overview

SQL Operators are VERY important for data filtering, backend validation, API response verification, and writing optimized SQL queries.

Topics Covered

- Arithmetic Operators ★
- Comparison Operators ★
- Logical Operators ★
- IN Operator ★
- BETWEEN Operator ★
- LIKE Operator ★
- IS NULL Operator ★

► Arithmetic Operators

What is it: Used for mathematical calculations.

Key Components:

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulo

How YOU should answer: Arithmetic operators perform calculations on numeric column values.

SQL

```
SELECT salary, salary * 1.1 AS increased_salary FROM employees;
```

► Logical Operators

What is it: Combine multiple conditions.

Key Components:

- AND — all conditions true
- OR — any condition true
- NOT — reverses condition

How YOU should answer: Logical operators build complex filter conditions.

SQL

```
SELECT * FROM orders WHERE status = 'PENDING' OR status = 'PROCESSING';
```

► IS NULL Operator

What is it: Check for NULL values.

Key Components:

- IS NULL — finds null records
- IS NOT NULL — finds non-null records

How YOU should answer: IS NULL and IS NOT NULL check for missing or existing values in columns.

SQL

```
SELECT * FROM customers WHERE phone IS NULL;
```

```
SELECT * FROM users WHERE email IS NOT NULL;
```

Common Mistakes

- ✗ Using = NULL instead of IS NULL
- ✗ Wrong operator for pattern matching
- ✗ Missing parentheses in complex conditions

Expected Interview Questions

- SQL operator types?
- IS NULL vs = NULL?
- Arithmetic operators in SQL?

ORDER BY CLAUSE

6. ORDER BY Clause ★

Overview

ORDER BY is one of the MOST IMPORTANT SQL concepts for sorting data, reporting queries, backend validation, and real-time database testing.

Topics Covered

- What is ORDER BY ★
- ASC Sorting ★
- DESC Sorting ★
- Multiple Column Sorting ★
- ORDER BY with WHERE ★
- ORDER BY with LIMIT ★

► What is ORDER BY

What is it: ORDER BY is used to sort query results.

Key Components:

- ASC — ascending (default)
- DESC — descending
- Multiple column sorting

How YOU should answer: ORDER BY sorts records in ascending or descending order based on specified columns.

SQL

```
SELECT * FROM employees ORDER BY salary DESC;  
SELECT * FROM users ORDER BY name ASC;
```

► Multiple Column Sorting

What is it: Sort by more than one column.

Key Components:

- Primary sort column
- Secondary sort column
- Mix ASC and DESC

How YOU should answer: Multiple column ORDER BY first sorts by the first column, then by subsequent columns.

SQL

```
SELECT * FROM employees ORDER BY department ASC, salary DESC;
```

► ORDER BY with LIMIT

What is it: Get top-N records efficiently.

Key Components:

- Top-N queries
- Pagination support

How YOU should answer: Combining ORDER BY with LIMIT retrieves top-N records efficiently.

SQL

```
-- Top 5 highest paid employees  
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 5;
```

Common Mistakes

- ✗ Wrong sorting expectations
- ✗ Forgetting default is ASC
- ✗ Not combining with LIMIT for top-N

Expected Interview Questions

- What is ORDER BY?
- ASC vs DESC?
- Multiple column sorting?
- ORDER BY with LIMIT example?

DISTINCT KEYWORD

7. DISTINCT Keyword ★

Overview

DISTINCT is one of the MOST IMPORTANT SQL concepts for removing duplicate records, data validation, reporting queries, and backend database testing.

Topics Covered

- What is DISTINCT ★
- DISTINCT on Single Column ★
- DISTINCT on Multiple Columns ★
- DISTINCT vs GROUP BY ★
- Performance Considerations ★

► What is DISTINCT

What is it: DISTINCT removes duplicate values from query results.

Key Components:

- Unique values only
- Duplicate elimination
- Applied to full selected row

How YOU should answer: DISTINCT removes duplicate rows from query results, returning only unique values.

SQL

```
SELECT DISTINCT city FROM employees;  
SELECT DISTINCT department FROM employees;
```

► DISTINCT on Multiple Columns

What is it: Remove duplicates across multiple columns.

Key Components:

- Combination uniqueness
- Multiple column check

How YOU should answer: DISTINCT on multiple columns removes rows where all specified columns are identical.

SQL

```
SELECT DISTINCT department, city FROM employees;
```

► DISTINCT vs GROUP BY

What is it: Two ways to get unique values.

Key Components:

- DISTINCT — simpler for unique values
- GROUP BY — needed with aggregate functions

How YOU should answer: DISTINCT is simpler for unique values while GROUP BY is used when aggregation is needed.

SQL

```
SELECT DISTINCT department FROM employees;
```

```
-- vs
```

```
SELECT department FROM employees GROUP BY department;
```

Common Mistakes

- ✗ Expecting full row uniqueness always
- ✗ Confusing with GROUP BY
- ✗ Performance on large datasets ignored

Expected Interview Questions

- What is DISTINCT?
- DISTINCT vs GROUP BY?
- Multiple column DISTINCT?
- Duplicate handling?

LIMIT CLAUSE

8. LIMIT Clause ★

Overview

LIMIT clause is one of the MOST IMPORTANT SQL concepts for pagination, top-N queries, performance optimization, and backend validation.

Topics Covered

- What is LIMIT ★
- Basic LIMIT Usage ★

- LIMIT with ORDER BY ★
- Pagination using LIMIT ★
- LIMIT with OFFSET ★
- Top-N Queries ★

► What is LIMIT

What is it: LIMIT restricts the number of rows returned by a query.

Key Components:

- Row restriction
- Pagination
- Performance optimization

How YOU should answer: LIMIT restricts how many rows are returned, useful for pagination and top-N queries.

SQL

```
SELECT * FROM employees LIMIT 10;
```

► Pagination using LIMIT and OFFSET

What is it: Navigate through large datasets page by page.

Key Components:

- LIMIT — rows per page
- OFFSET — rows to skip

How YOU should answer: LIMIT with OFFSET implements pagination by skipping rows and returning a specific page.

SQL

```
-- Page 1
SELECT * FROM employees LIMIT 10 OFFSET 0;

-- Page 2
SELECT * FROM employees LIMIT 10 OFFSET 10;
```

► Top-N Queries

What is it: Get top records efficiently.

Key Components:

- Combine with ORDER BY
- Performance efficient

How YOU should answer: LIMIT with ORDER BY efficiently retrieves the top-N records by a specific column.

SQL

```
SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 3;
```

Common Mistakes

- ✗ Database syntax differences ignored (MySQL vs Oracle)
- ✗ Missing ORDER BY with LIMIT
- ✗ Ignoring OFFSET for pagination

Expected Interview Questions

- What is LIMIT clause?
- Pagination with LIMIT and OFFSET?
- Top-N query example?

ALIASES IN SQL

9. Aliases in SQL ★

Overview

Aliases are VERY important for query readability, complex SQL queries, joins and reporting, and backend validation queries.

Topics Covered

- What are Aliases ★
- Column Aliases ★
- Table Aliases ★
- Aliases with Expressions ★
- Aliases with Joins ★
- Aliases with Aggregate Functions ★

► What are Aliases

What is it: Aliases are temporary names assigned to columns or tables.

Key Components:

- Temporary naming
- Better readability
- Query simplification

How YOU should answer: Aliases provide temporary names for columns and tables to improve query readability.

```
SQL
```

```
SELECT name AS employee_name, salary AS monthly_salary FROM employees AS emp;
```

► Column Aliases

What is it: Rename column output for readability.

Key Components:

- AS keyword
- Readable output
- Calculation naming

How YOU should answer: Column aliases rename query output columns using AS keyword.

SQL

```
SELECT salary * 12 AS annual_salary, name AS full_name FROM employees;
```

► Table Aliases

What is it: Shorter names for table references.

Key Components:

- Shorter reference
- Essential for self-joins
- Multi-table queries

How YOU should answer: Table aliases provide shorter names for tables especially useful in join queries.

SQL

```
SELECT e.name, d.department_name  
FROM employees e  
JOIN departments d ON e.dept_id = d.id;
```

► Aliases with Aggregate Functions

What is it: Name aggregate results meaningfully.

Key Components:

- COUNT AS total
- AVG AS average
- SUM AS total

How YOU should answer: Aliases make aggregate function results readable in reports.

SQL

```
SELECT department, COUNT(*) AS total_employees, AVG(salary) AS avg_salary  
FROM employees GROUP BY department;
```

Common Mistakes

- ✗ Confusing permanent rename with alias
- ✗ Using aliases in WHERE clause (use HAVING instead)
- ✗ Missing AS keyword

Expected Interview Questions

- What are aliases?

- Column vs table aliases?
- Why aliases used in joins?
- Real-time examples?

INSERT STATEMENT

10. INSERT Statement ★

Overview

INSERT is one of the MOST IMPORTANT SQL concepts for adding new records, test data creation, backend validation, and API testing scenarios.

Topics Covered

- What is INSERT ★
- Basic INSERT Syntax ★
- INSERT into Specific Columns ★
- INSERT Multiple Rows ★
- INSERT with NULL Values ★
- INSERT from Another Table ★

► What is INSERT

What is it: INSERT statement adds new records into a database table.

Key Components:

- Add new records
- Test data creation
- DML operation

How YOU should answer: INSERT statement is used to add new rows of data into database tables.

SQL

```
INSERT INTO employees (name, salary, department) VALUES ('Kalpesh', 75000, 'IT');
```

► INSERT Multiple Rows

What is it: Add multiple records in one statement.

Key Components:

- Batch insertion
- Performance efficient
- Reduced round trips

How YOU should answer: INSERT with multiple value sets adds several records in a single query.

SQL

```
INSERT INTO employees (name, salary)
VALUES ('Alice', 60000), ('Bob', 65000), ('Charlie', 70000);
```

► INSERT from Another Table

What is it: Copy data between tables.

Key Components:

- INSERT INTO ... SELECT
- Data migration
- Backup tables

How YOU should answer: INSERT INTO ... SELECT copies data from one table into another.

```
SQL
INSERT INTO archived_employees SELECT * FROM employees WHERE status = 'INACTIVE';
```

► Real-Time Testing Scenarios

What is it: INSERT in SDET testing.

Key Components:

- Create test users
- Seed test data
- Setup test environment

How YOU should answer: INSERT is used to create test data for automated testing scenarios.

```
SQL
INSERT INTO users (email, password, role) VALUES ('test@auto.com', 'pass123',
'TESTER');
```

Common Mistakes

- ✗ Missing mandatory columns
- ✗ Wrong data types
- ✗ Not using transactions for multiple inserts

Expected Interview Questions

- What is INSERT statement?
- INSERT multiple rows syntax?
- INSERT from another table?

UPDATE STATEMENT

11. UPDATE Statement ★

Overview

UPDATE is one of the MOST IMPORTANT SQL concepts for modifying existing records, backend validation, data correction, and real-time database operations.

Topics Covered

- What is UPDATE ★
- UPDATE with WHERE ★
- UPDATE Multiple Columns ★
- UPDATE using Conditions ★
- UPDATE Best Practices ★

► What is UPDATE

What is it: UPDATE statement modifies existing records in a database table.

Key Components:

- Modify existing records
- DML operation
- Conditional updates

How YOU should answer: UPDATE statement changes existing data in one or more columns of database records.

SQL

```
UPDATE employees SET salary = 80000 WHERE name = 'Kalpesh';
```

► UPDATE with WHERE

What is it: Always use WHERE to avoid updating all records.

Key Components:

- Conditional update
- Specific record modification
- Prevent full table update

How YOU should answer: WHERE clause in UPDATE is critical to target specific records and avoid updating all rows.

SQL

```
UPDATE employees SET department = 'Engineering', salary = 85000 WHERE id = 101;
```

► Real-Time Testing Scenarios

What is it: UPDATE in SDET testing.

Key Components:

- Reset test data
- Update test user status
- Data correction after test

How YOU should answer: UPDATE is used to reset test data, modify user statuses, and prepare test environments.

SQL

```
UPDATE orders SET status = 'PENDING' WHERE test_flag = TRUE;
```

Common Mistakes

- ✗ Missing WHERE clause — updates ALL rows
- ✗ Not verifying before updating
- ✗ Not using transactions

Expected Interview Questions

- What is UPDATE statement?
- UPDATE with WHERE example?
- Update multiple columns?

DELETE STATEMENT

12. DELETE Statement ★

Overview

DELETE is one of the MOST IMPORTANT SQL concepts for removing records, data cleanup, backend validation, and database maintenance.

Topics Covered

- What is DELETE ★
- DELETE with WHERE ★
- DELETE vs TRUNCATE ★
- DELETE Best Practices ★
- Real-Time Testing Scenarios ★

► What is DELETE

What is it: DELETE statement removes records from a database table.

Key Components:

- Remove records
- DML operation
- Conditional deletion
- Rollback possible

How YOU should answer: DELETE removes specific records from a table based on WHERE conditions.

SQL

```
DELETE FROM employees WHERE id = 101;
```

► DELETE with WHERE

What is it: Use WHERE to delete specific records.

Key Components:

- Targeted deletion
- Prevent full table wipe

How YOU should answer: WHERE clause in DELETE targets specific records for removal.

```
SQL
```

```
DELETE FROM orders WHERE status = 'CANCELLED' AND created_at < '2023-01-01';
```

► Real-Time Testing Scenarios

What is it: DELETE in SDET testing.

Key Components:

- Clean test data
- Remove test records
- Post-test cleanup

How YOU should answer: DELETE is used after test execution to clean up test data.

```
SQL
```

```
DELETE FROM users WHERE email LIKE '%@test.com';
```

Common Mistakes

- ✗ Missing WHERE clause — deletes ALL rows
- ✗ Not taking backup before bulk delete
- ✗ Confusing DELETE and TRUNCATE

Expected Interview Questions

- What is DELETE statement?
- DELETE vs TRUNCATE?
- Safe delete practices?

Feature	DELETE	TRUNCATE
Removes	Specific rows (WHERE)	All rows
Speed	Slower	Faster
WHERE clause	Supported	Not supported
Logging	Row-level	Minimal
Rollback	Possible	DB-specific
Identity reset	Usually no	Usually yes

TRUNCATE STATEMENT

13. TRUNCATE Statement ★

Overview

TRUNCATE is one of the MOST IMPORTANT SQL concepts for fast data removal, table cleanup, test environment reset, and performance optimization.

Topics Covered

- What is TRUNCATE ★
- TRUNCATE Syntax ★
- TRUNCATE vs DELETE ★
- Performance Benefits ★
- Identity Reset Behavior ★
- Transaction Considerations ★
- Real-Time Testing Scenarios ★

► What is TRUNCATE

What is it: TRUNCATE removes all records from a table quickly.

Key Components:

- Full table cleanup
- Fast execution
- Table reset
- No WHERE support

How YOU should answer: TRUNCATE statement removes all records from a table efficiently without deleting the structure.

```
SQL
```

```
TRUNCATE TABLE orders;
```

► TRUNCATE vs DELETE

What is it: Key differences between the two removal operations.

Key Components:

- TRUNCATE — all rows, fast, minimal logging, resets identity
- DELETE — conditional, row-level logging, slower, rollback possible

How YOU should answer: TRUNCATE removes all rows faster with minimal logging while DELETE supports conditional removal.

► Performance Benefits

What is it: TRUNCATE is faster than DELETE.

Key Components:

- Minimal logging
- Faster execution
- Better efficiency

How YOU should answer: TRUNCATE performs faster because it removes entire data pages rather than individual rows.

SQL

```
-- Reset test environment quickly  
TRUNCATE TABLE test_data;
```

► Real-Time Testing Scenarios

What is it: TRUNCATE in SDET testing.

Key Components:

- Reset test environment data
- Clear temporary transaction logs
- Reset staging tables

How YOU should answer: TRUNCATE is used to quickly reset test environment data between test runs.

SQL

```
TRUNCATE TABLE staging_orders;
```

Common Mistakes

- ✗ Confusing TRUNCATE with DELETE
- ✗ Using TRUNCATE in production without backup
- ✗ Ignoring identity reset

Expected Interview Questions

- TRUNCATE vs DELETE?
- Why TRUNCATE is faster?
- When to use TRUNCATE?
- Identity reset behavior?

DROP STATEMENT

14. DROP Statement ★

Overview

DROP is one of the MOST IMPORTANT SQL concepts for removing database objects, database cleanup, schema management, and test environment maintenance.

Topics Covered

- What is DROP ★
- DROP TABLE ★
- DROP DATABASE ★
- DROP vs DELETE vs TRUNCATE ★
- Dependency Considerations ★
- DROP Best Practices ★

► What is DROP

What is it: DROP permanently removes database objects like tables, databases, or columns.

Key Components:

- Object removal
- Permanent deletion
- Schema cleanup

How YOU should answer: DROP statement permanently removes database objects. Unlike DELETE and TRUNCATE, it also removes the structure.

```
SQL
```

```
DROP TABLE orders;
```

```
DROP DATABASE test_db;
```

► DROP vs DELETE vs TRUNCATE

What is it: Three different levels of removal.

Key Components:

- DROP — removes object + structure
- DELETE — removes specific rows
- TRUNCATE — removes all rows fast

How YOU should answer: DROP removes the entire object, TRUNCATE removes all rows fast, DELETE removes specific rows conditionally.

► Dependency Considerations

What is it: Database objects may depend on each other.

Key Components:

- Foreign key dependencies
- Constraint impact

How YOU should answer: DROP operations may fail if dependent objects or constraints exist on the table.

```
SQL
```

```
-- Check dependencies before DROP
```

```
SELECT * FROM information_schema.TABLE_CONSTRAINTS
```

```
WHERE TABLE_NAME = 'employees';
```

Common Mistakes

- ✗ Dropping wrong objects
- ✗ Not checking dependencies
- ✗ No backup before DROP

Expected Interview Questions

- DROP TABLE vs DELETE?
- DROP vs TRUNCATE vs DELETE?
- How to handle dependencies?

Feature	DROP	DELETE	TRUNCATE
Removes	Object+structure	Specific rows	All rows
Rollback	No	Yes	DB-specific
WHERE	Not applicable	Yes	No
Structure	Deleted	Remains	Remains

ALTER TABLE

15. ALTER TABLE ★

Overview

ALTER TABLE is one of the MOST IMPORTANT SQL concepts for modifying table structure, schema updates, backend database changes, and production maintenance.

Topics Covered

- What is ALTER TABLE ★
- ADD COLUMN ★
- MODIFY COLUMN ★
- DROP COLUMN ★
- RENAME COLUMN ★
- ALTER TABLE Best Practices ★

► What is ALTER TABLE

What is it: ALTER TABLE is used to modify the structure of an existing database table.

Key Components:

- Schema modification
- Structure updates

- Table maintenance

How YOU should answer: ALTER TABLE modifies existing table structure by adding, modifying, dropping, or renaming columns.

SQL

```
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);
```

► ADD COLUMN

What is it: Adds new columns to existing tables.

Key Components:

- Schema extension
- New field addition

How YOU should answer: ADD COLUMN is used to add new fields into existing tables without recreating them.

SQL

```
ALTER TABLE employees ADD COLUMN bonus DECIMAL(10,2);
```

► MODIFY COLUMN

What is it: Changes datatype or size of columns.

Key Components:

- Datatype modification
- Size changes

How YOU should answer: MODIFY COLUMN changes existing column definitions such as datatype or size.

SQL

```
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(15,2);
```

► DROP COLUMN

What is it: Removes columns from tables.

Key Components:

- Column cleanup
- Schema simplification

How YOU should answer: DROP COLUMN removes unnecessary columns permanently from a table structure.

SQL

```
ALTER TABLE employees DROP COLUMN bonus;
```

► RENAME COLUMN

What is it: Changes existing column names.

Key Components:

- Schema readability
- Naming consistency

How YOU should answer: RENAME COLUMN changes column names for better readability or business alignment.

SQL

```
ALTER TABLE employees RENAME COLUMN emp_name TO full_name;
```

Common Mistakes

- ✗ Wrong schema modifications
- ✗ Breaking dependencies
- ✗ Missing backups before ALTER

Expected Interview Questions

- What is ALTER TABLE?
- ADD COLUMN vs MODIFY COLUMN?
- How to rename a column?
- Best practices for schema changes?

NULL HANDLING

16. NULL Handling ★

Overview

NULL handling is one of the MOST IMPORTANT SQL concepts for missing data management, backend validation, real-world database handling, and API response validation.

Topics Covered

- What is NULL ★
- IS NULL ★
- IS NOT NULL ★
- NULL vs Zero vs Empty String ★
- NULL with Aggregate Functions ★
- NULL Handling Functions ★

► What is NULL

What is it: NULL in SQL represents missing, unknown, or unavailable data.

Key Components:

- Missing value

- Unknown data
- Requires IS NULL check

How YOU should answer: NULL represents missing or unavailable data. It is not zero or empty string.

SQL

```
SELECT * FROM employees WHERE email IS NULL;
```

► IS NULL and IS NOT NULL

What is it: Check for presence or absence of values.

Key Components:

- IS NULL — finds null records
- IS NOT NULL — validates data presence

How YOU should answer: IS NULL checks for missing data while IS NOT NULL validates data presence.

SQL

```
SELECT * FROM employees WHERE phone IS NULL;
```

```
SELECT * FROM employees WHERE email IS NOT NULL;
```

► NULL vs Zero vs Empty String

What is it: Common confusion in SQL.

Key Components:

- NULL — missing/unknown value
- Zero (0) — actual numeric value
- Empty string ("") — actual blank text

How YOU should answer: NULL represents unknown data, while zero and empty string are actual stored values.

SQL

```
SELECT * FROM users WHERE salary IS NULL; -- missing
```

```
SELECT * FROM users WHERE salary = 0; -- zero salary
```

```
SELECT * FROM users WHERE name = ''; -- blank name
```

► NULL Handling Functions

What is it: Functions to replace NULL with default values.

Key Components:

- COALESCE — returns first non-null value
- IFNULL — MySQL NULL replacement

How YOU should answer: COALESCE and IFNULL replace NULL values with meaningful default values.

SQL

```
SELECT COALESCE(phone, 'Not Available') AS contact FROM employees;
```

```
SELECT IFNULL(bonus, 0) AS bonus FROM employees;
```

► NULL with Aggregate Functions

What is it: How aggregate functions handle NULL.

Key Components:

- COUNT(*) — includes NULL rows
- COUNT(column) — ignores NULL
- SUM/AVG/MIN/MAX — ignore NULL

How YOU should answer: Most aggregate functions ignore NULL values except COUNT(*) which counts all rows.

SQL

```
SELECT COUNT(*) AS total_rows FROM employees;
```

```
SELECT COUNT(email) AS with_email FROM employees;
```

Common Mistakes

- ✗ Using = NULL instead of IS NULL
- ✗ Confusing NULL with empty string
- ✗ Not handling NULL in aggregates

Expected Interview Questions

- What is NULL in SQL?
- IS NULL vs = NULL?
- NULL vs empty string vs zero?
- COALESCE vs IFNULL?
- How aggregate functions handle NULL?

★ NULL Handling Interview Gold

1. IS NULL — correct way to check NULL
2. COALESCE — replace NULL with default
3. COUNT(*) vs COUNT(column)
4. NULL vs Zero vs Empty String
5. Real-time: NULL in optional API response fields

SQL CONSTRAINTS

17. SQL Constraints ★

Overview

SQL Constraints are one of the MOST IMPORTANT concepts for data integrity, schema design, backend validation, and database testing.

Topics Covered

- What are SQL Constraints ★
- NOT NULL Constraint ★
- UNIQUE Constraint ★
- PRIMARY KEY Constraint ★
- FOREIGN KEY Constraint ★
- CHECK Constraint ★
- DEFAULT Constraint ★

► What are SQL Constraints

What is it: Constraints are rules applied on database columns to enforce data integrity.

Key Components:

- Data validation
- Integrity enforcement
- Business rules

How YOU should answer: SQL constraints are rules used to maintain data accuracy and integrity in database tables.

SQL

```
CREATE TABLE employees (  
  id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE  
);
```

► NOT NULL Constraint

What is it: Ensures a column cannot have NULL values.

Key Components:

- Mandatory fields
- Data completeness

How YOU should answer: NOT NULL constraint ensures critical columns always have values.

SQL

```
name VARCHAR(100) NOT NULL,  
email VARCHAR(100) NOT NULL
```

► UNIQUE Constraint

What is it: Ensures all values in a column are different.

Key Components:

- No duplicate values
- Multiple per table
- Allows one NULL

How YOU should answer: UNIQUE constraint prevents duplicate values in a column.

SQL

```
email VARCHAR(100) UNIQUE
```

► **CHECK Constraint**

What is it: Validates column values against a condition.

Key Components:

- Value range validation
- Business rule enforcement

How YOU should answer: CHECK constraint ensures column values satisfy specified conditions.

SQL

```
age INT CHECK (age >= 18),  
salary DECIMAL CHECK (salary > 0)
```

► **DEFAULT Constraint**

What is it: Sets a default value when none is provided.

Key Components:

- Auto-fill on insert
- Default values

How YOU should answer: DEFAULT constraint automatically assigns a value when INSERT does not provide one.

SQL

```
status VARCHAR(20) DEFAULT 'PENDING',  
created_at DATETIME DEFAULT NOW()
```

Common Mistakes

- ✗ Ignoring constraint rules
- ✗ Weak schema design
- ✗ Not testing constraint violations

Expected Interview Questions

- What are SQL constraints?
- NOT NULL vs UNIQUE?

- CHECK constraint example?
- Real-time constraint testing?

Constraint	Purpose	Example
NOT NULL	No missing values	name NOT NULL
UNIQUE	No duplicates	email UNIQUE
PRIMARY KEY	Unique identifier	id PRIMARY KEY
FOREIGN KEY	Table relationship	dept_id FOREIGN KEY
CHECK	Value validation	age CHECK (age >= 18)
DEFAULT	Auto value	status DEFAULT 'ACTIVE'

PRIMARY KEY VS FOREIGN KEY

18. Primary Key vs Foreign Key ★

Overview

Primary Key and Foreign Key are one of the MOST IMPORTANT SQL concepts for database design, referential integrity, backend validation, and SDET interviews.

Topics Covered

- What is Primary Key ★
- What is Foreign Key ★
- Why Keys are Important ★
- Primary Key Features ★
- Foreign Key Features ★
- Table Relationships ★
- Real-Time Testing Scenarios ★

► What is Primary Key

What is it: Primary Key uniquely identifies each record in a table.

Key Components:

- Unique values only
- Cannot be NULL
- One per table
- Auto-increment common

How YOU should answer: Primary Key is a constraint that uniquely identifies every row in a database table.

```
CREATE TABLE employees (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL  
);
```

► What is Foreign Key

What is it: Foreign Key creates a relationship between two tables.

Key Components:

- References Primary Key of another table
- Ensures referential integrity
- Can have duplicates
- Can be NULL

How YOU should answer: Foreign Key links two tables together by referencing the Primary Key of a related table.

```
SQL  
  
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  employee_id INT,  
  FOREIGN KEY (employee_id) REFERENCES employees(id)  
);
```

► Table Relationships

What is it: Foreign Keys enable parent-child table relationships.

Key Components:

- One-to-One
- One-to-Many (most common)
- Many-to-Many

How YOU should answer: Foreign Keys establish relationships between tables allowing JOIN operations.

```
SQL  
  
SELECT e.name, o.order_id  
FROM employees e  
JOIN orders o ON e.id = o.employee_id;
```

► Real-Time Testing Scenarios

What is it: Keys validation in SDET.

Key Components:

- Validate referential integrity
- Orphan record detection

How YOU should answer: Keys are tested to ensure no orphan records exist and referential integrity is maintained.

SQL

```
-- Check for orphan orders  
SELECT * FROM orders  
WHERE employee_id NOT IN (SELECT id FROM employees);
```

Common Mistakes

- ✗ Confusing Primary and Foreign Key
- ✗ Not checking referential integrity
- ✗ Ignoring orphan records

Expected Interview Questions

- Difference between Primary Key and Foreign Key?
- Can Primary Key be NULL?
- Can Foreign Key be NULL?
- What is referential integrity?

Feature	Primary Key	Foreign Key
Purpose	Uniquely identify row	Link two tables
Duplicates	Not allowed	Allowed
NULL values	Not allowed	Allowed
Count per table	One	Multiple
References	Self	Other table PK

★ Final Interview Gold — Part 1

1. SQL Commands: DDL vs DML vs DQL vs TCL
2. NULL Handling: IS NULL, COALESCE, NULL vs Zero
3. DELETE vs TRUNCATE vs DROP — 3-way comparison
4. Constraints: NOT NULL, UNIQUE, CHECK, DEFAULT
5. Primary Key vs Foreign Key — referential integrity
6. ALTER TABLE: ADD, MODIFY, DROP, RENAME COLUMN